

HW 1:

Part 1 - Binary LEDs:

```
const int buttonPin = 2;

const int ledPins[] = {9, 10, 11, 12, LED_BUILTIN}; // Array of LED pins
const int numLeds = sizeof(ledPins) / sizeof(ledPins[0]); // Calculate number of LEDs
boolean buttonState = LOW;
int pressCount = 0;
unsigned long lastPressTime = 0;
const unsigned long debounceDelay = 100;
const unsigned long pressTimeout = 2000; // 2 seconds
const unsigned long ledOnDuration = 2000; // 2 seconds

void setup() {
    Serial.begin(9600); // Initialize serial communication
    pinMode(buttonPin, INPUT_PULLUP); // Set button pin as input with internal pull-up
    resistor
    for (int i = 0; i < numLeds; i++) {
        pinMode(ledPins[i], OUTPUT); // Set LED pins as output
    }
}

void loop() {
    // Read the state of the button with debounce
    boolean reading = digitalRead(buttonPin);
    if (reading != buttonState) {
        delay(debounceDelay);
        reading = digitalRead(buttonPin);
        if (reading != buttonState) buttonState = reading;
    }

    // Check if the button is pressed
    if (buttonState == LOW && (millis() - lastPressTime) > debounceDelay) {
        // Increment the press count
        pressCount++;
        lastPressTime = millis(); // Update last press time

        // Print the updated count on the serial monitor
        Serial.print("Button pressed, count: ");
        Serial.println(pressCount);

        // Reset lastPressTime to start detecting button presses again
        lastPressTime = millis();
    }
}
```

```

    }

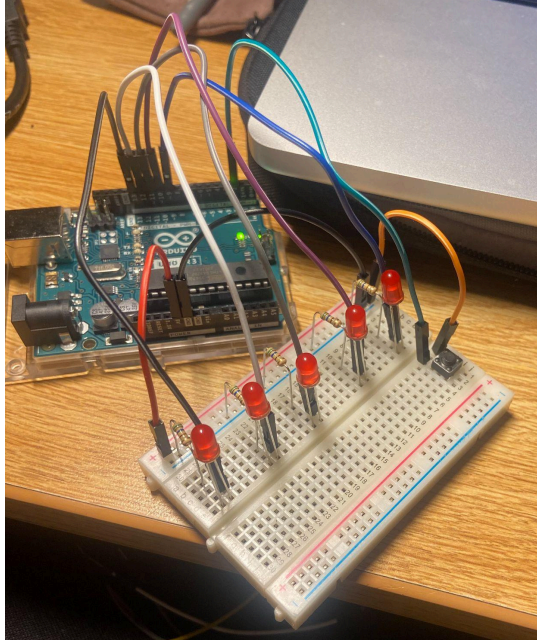
    // Check if press timeout has elapsed
    if (millis() - lastPressTime > pressTimeout && pressCount > 0) {
        // Perform action after timeout (e.g., light up LEDs according to binary
representation)
        int num = pressCount;
        for (int i = 0; i < numLeds; i++) {
            int bit = num & 1; // Extract the least significant bit
            digitalWrite(ledPins[i], bit); // Turn on/off LED based on binary
representation
            num >>= 1; // Shift right to check the next bit
        }
        delay(ledOnDuration); // Delay to keep the LEDs on for 2 seconds
        // Turn off all LEDs
        for (int i = 0; i < numLeds; i++) {
            digitalWrite(ledPins[i], LOW);
        }
        // Reset press count
        pressCount = 0;
    }
}

```

Write up:

I made this code show me the number of times I was pressing the button (using the serial monitor) and had it utilize a variety of functions that allowed for a press counter, which added each time the button was pressed, and after there were no clicks for 2 seconds (a pressTimeout) the value (number of clicks) was accepted and then represented as its binary number. The least significant bit was extracted and the LEDs then represented it by having the pins iterate in a for loop, starting from the leftmost pin, LED_BUILTIN (13), and then making its way to the rightmost, pin 9, and using the binary representation, either set the LED to HIGH (on) or LOW (off) based on if the remainder was a non-zero integer or not and worked its way down through the remaining binary numbers as they are assigned to each LED from left to right. The LEDs were then coded to stay on for 2 seconds and then all were turned off. Lastly, the count was then reset and awaited new input values (button clicks).

Picture:



Part 2 - Joystick Game:

```
void setup() {  
    // put your setup code here, to run once:  
    Serial.begin(9600);  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
    // Define parameters for the sine function  
    float amplitude = 500.0; // Amplitude of the sine wave  
    float frequency = 0.1; // Frequency of the sine wave in Hz  
    float phase = 0.0; // Phase shift of the sine wave  
    float samplingInterval = 0.1; // Time between samples in seconds  
  
    // Loop to generate and send sine wave data  
    for (float t = 0.0; t < 10.0; t += samplingInterval) {  
        // Calculate sine wave value at time t  
        float sinValue = amplitude * sin(2 * PI * frequency * t + phase)+250;  
        float sinValue2 = sinValue+500;  
  
        // Send time and sine value over serial  
        //Serial.print(t);  
        int value = analogRead(A0);  
        Serial.print(value);  
    }
```

```

Serial.print(",");
Serial.print(sinValue);

Serial.print(",");
Serial.println(sinValue2);

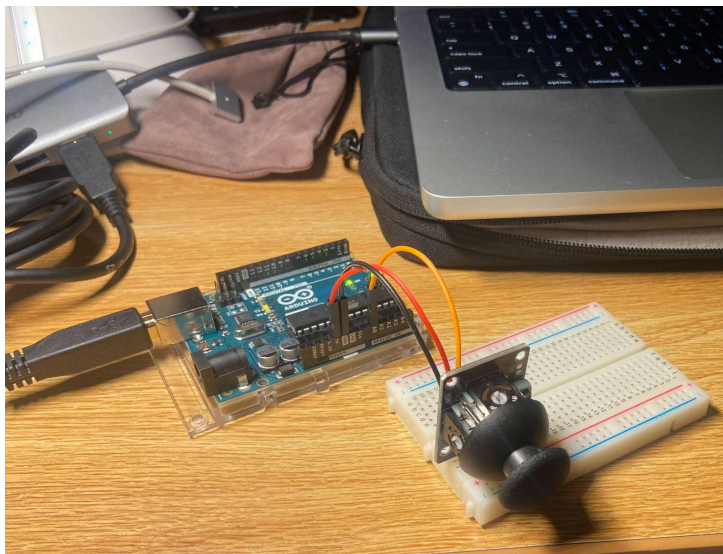
if (value >= sinValue2 || value <= sinValue) { // If joysticks crosses the line the
game end
    Serial.println("Game Over!");
    delay(1000);
    break;
}
// Delay for sampling interval
delay(samplingInterval * 1000); // Convert sampling interval to milliseconds
}
}

```

Write up:

I made a code that plotted two identical sine functions 250 units apart and this gave me a changing boundary that I needed to stay within. I made it so that the value I was starting at, 506-507 when my joystick was at rest, was in the middle of the two limits to start with. I also made it so that when you went above or below the sine functions (the game's boundaries) it would print "Game Over!" and break the for loop, ultimately restarting the game to the original starting point and then playing the game again in the serial monitor.

Picture:





HW 2

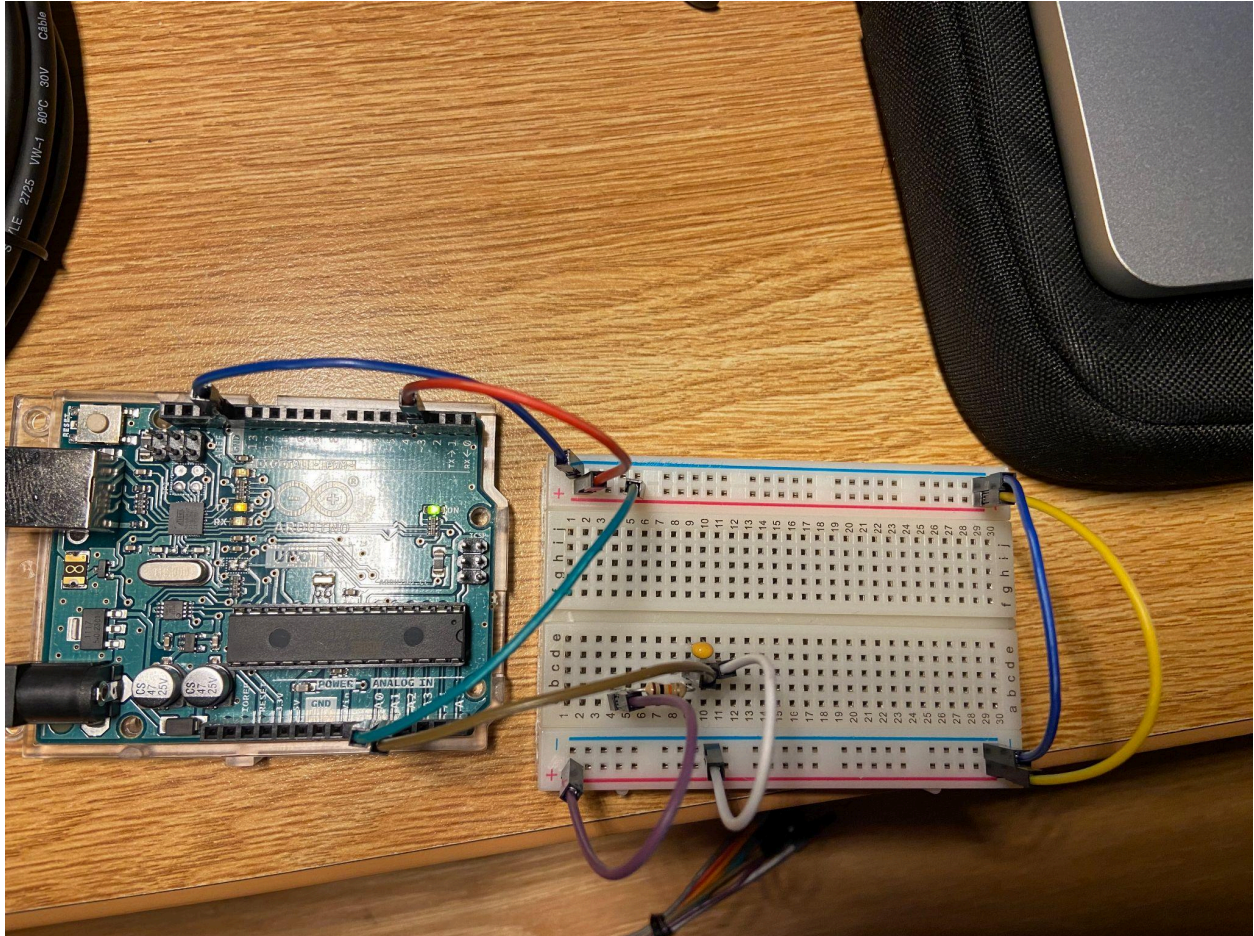
PART 1 - PWM SIN WAVE WITH FILTER:

```
void setup() {  
    // put your setup code here, to run once:  
    Serial.begin(9600);  
    pinMode(A0, INPUT);  
    pinMode(A1, INPUT);  
    pinMode(3, OUTPUT);  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
    int pwmvoltagebeforefilter, pwmvoltageafterfilter;  
    for (int i=0; i<360; i++){  
        pwmvoltagebeforefilter = analogRead(A0);  
        pwmvoltageafterfilter = analogRead(A1);  
        int sinval = 128+127*sin(i*10*PI/180.0);  
        analogWrite(3,sinval);  
        Serial.print(sinval);  
        Serial.print(" ");  
        Serial.print(pwmvoltagebeforefilter);  
        Serial.print(" ");  
        Serial.println(pwmvoltageafterfilter);  
    }  
  
    Serial.print("0 1023 ");  
    Serial.print(analogRead(A0)); //Voltage Before Filter  
    Serial.print(" ");  
    Serial.println(analogRead(A1)); //Voltage After Filter  
  
    delay(100);  
}
```

Write Up:

I first assigned the input and output slots accordingly. Then I established the int for my variables for “pwmvoltagebeforefilter” and “pwmvoltageafterfilter” which declares these as integer values. Then established another integer variable for “sinval” and gets a new value for the sin wave every iteration. I then wrote up the code to allow for the PWM pin to produce a voltage for the sin wave for the analog inputs to read and store. This was then printed onto the serial potter so we could see what was being read by our analog inputs by our PWM voltage (shown below).

Pictures:





PART 2 - THERMISTOR (NON AMPLIFIED VOLTAGE DIVIDER):

```
const float R1 = 10000.0;
const float R0 = 10000.0;    // Resistance of thermistor at 25°C (10kΩ)
const float T0 = 25.0;      // Temperature at R0 (25°C)
const float B = 3950.0;     // Beta value

void setup() {
  Serial.begin(9600); // Initialize serial communication
}

void loop() {
  // ADC value from the thermistor
  int adcthermval = analogRead(A0);

  // Convert raw value to voltage
  float voltage = adcthermval * (5.0 / 1023.0); // Assuming a 5V reference voltage

  // Calculate resistance using voltage divider formula
  float resistance = R1 / ((5.0 / voltage) - 1.0); // R1 / (Vcc / Vout - 1)

  // Calculate temperature using Steinhart-Hart equation (coded below)
  float temp = calculateTemperature(resistance);

  // Print temperature to serial monitor
  Serial.print("Temperature: ");
```

```

Serial.print(temp);
Serial.println(" °C");

// Delay before taking another reading
delay(1000);
}

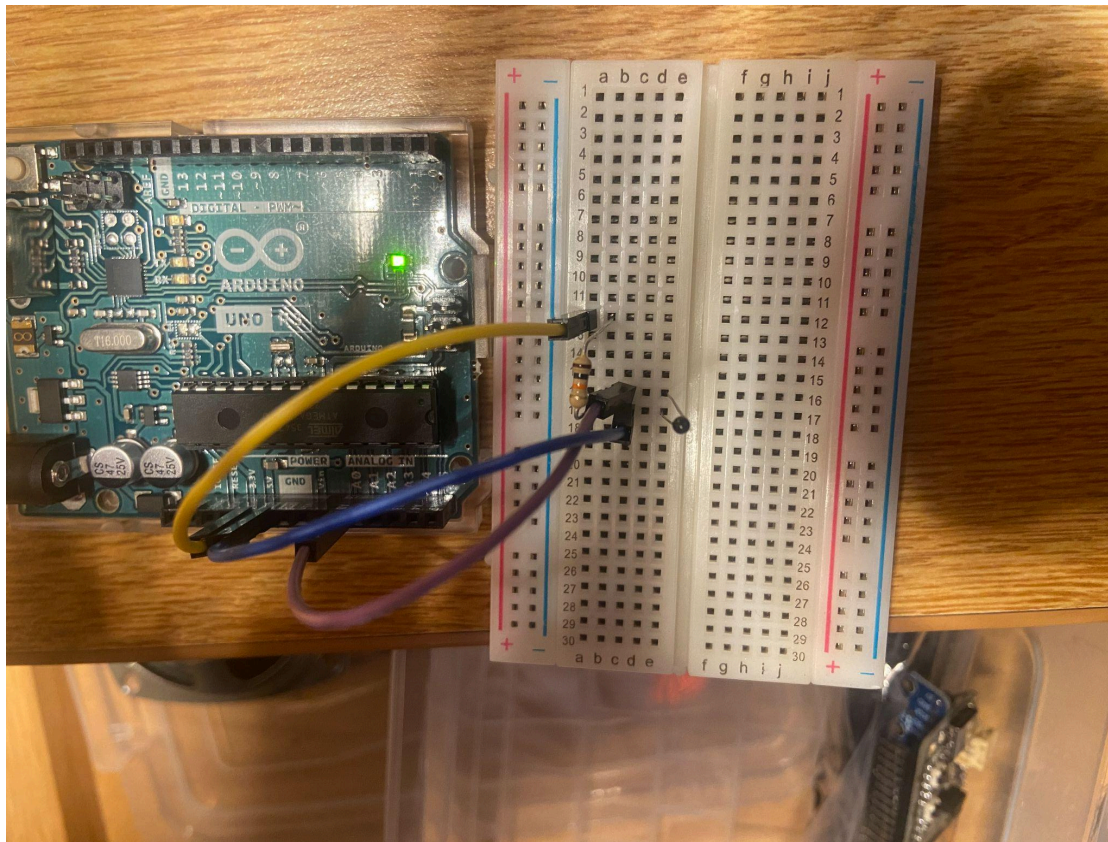
float calculateTemperature(float resistance) {
    // Calculate temperature using Steinhart-Hart equation for NTC thermistors
    float steinhart = log(resistance / R0);
    steinhart /= B;
    steinhart += 1.0 / (T0 + 273.15);
    steinhart = 1.0 / steinhart;
    steinhart -= 273.15;
    return steinhart;
}

```

Write Up:

I utilized the data sheet on the thermistor to tell me the Beta and R-value at 25 Celcius and that allowed the use of the Steinhart-Hart Equation giving us our temperature readings. I coded this to calculate the ADC value of the A0 input (Vout) which I then converted into a real voltage and used to calculate the resistance with the voltage divider formula, and finally used the Steinhart-Hart eqn to find the temperature.

Pictures:



PART 3 - THERMISTOR (AMPLIFIED VOLTAGE DIVIDER WITH OP AMP):

```
const float R1 = 10000.0;
const float R0 = 10000.0;      // Resistance of thermistor at 25°C (10kΩ)
const float T0 = 25.0;        // Temperature at R0 (25°C)
const float B = 3950.0;       // Beta value

void setup() {
  Serial.begin(9600); // Initialize serial communication
}

void loop() {
  // ADC value from the thermistor
  int adcthermval = analogRead(A0);
  int adcthermvalamplified = analogRead(A1);

  // Convert raw value to voltage
  float voltage = adcthermval * (5.0 / 1023.0); // Assuming a 5V reference voltage
  float voltageamplified = adcthermvalamplified * (5.0 / 1023.0); // Assuming a 5V
reference voltage

  // Calculate resistance using voltage divider formula
  float resistance = R1 / ((5.0 / voltage) - 1.0); // R1 / (Vcc / Vout - 1)
  float amplifiedresistance = R1 / ((5.0 / voltageamplified) - 1.0); // R1 / (Vcc /
Vout - 1)

  // Calculate temperature using Steinhart-Hart equation (coded below)
  float temp = calculateTemperature(resistance);
  float amplifiedtemp = calculateAmpTemperature(amplifiedresistance);

  // Print temperature to serial plotter
  Serial.print(temp);
  Serial.print(" ");

  Serial.print(amplifiedtemp);
  Serial.println(" ");

  // Delay before taking another reading
  delay(1000);
}

float calculateTemperature(float resistance) {
```

```

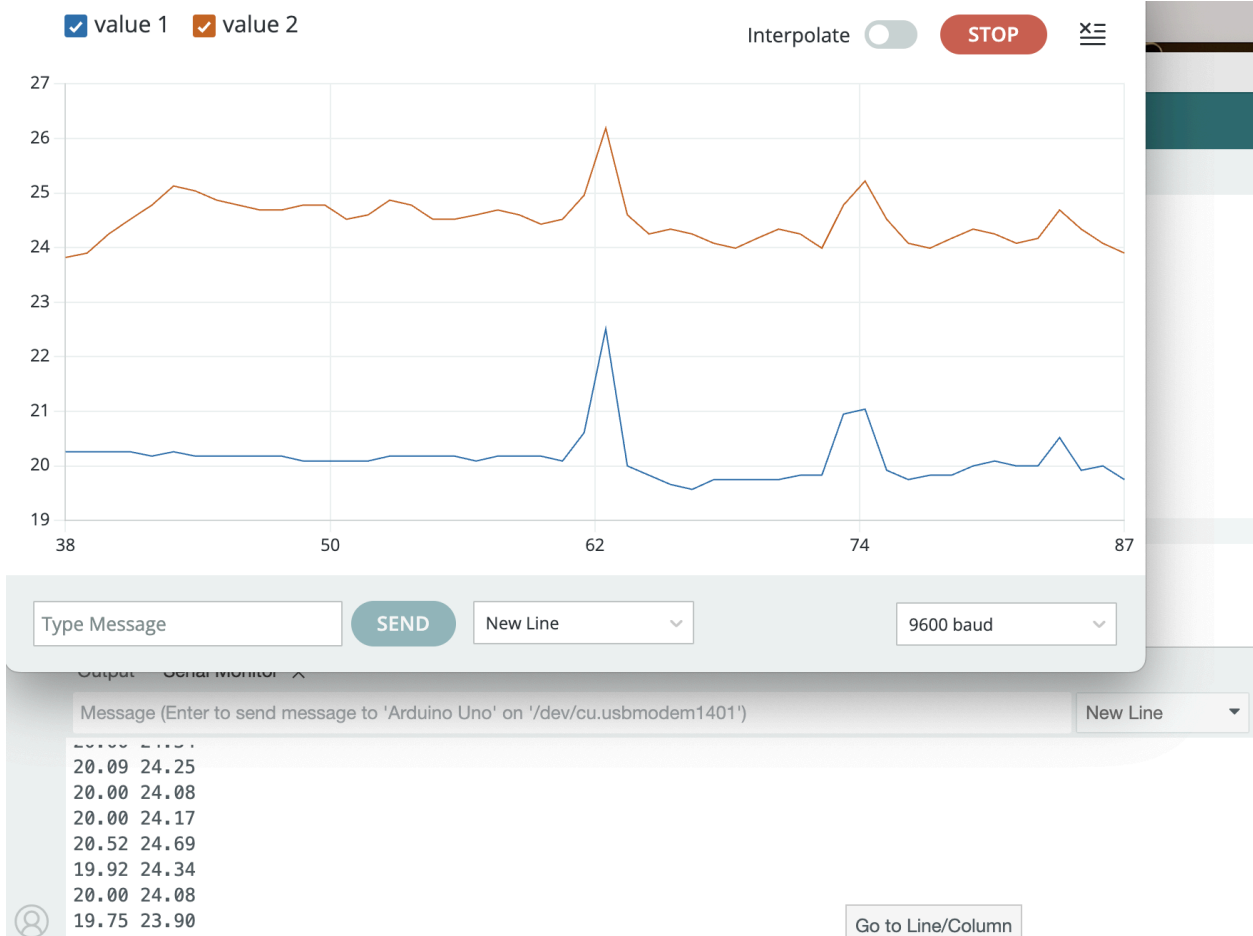
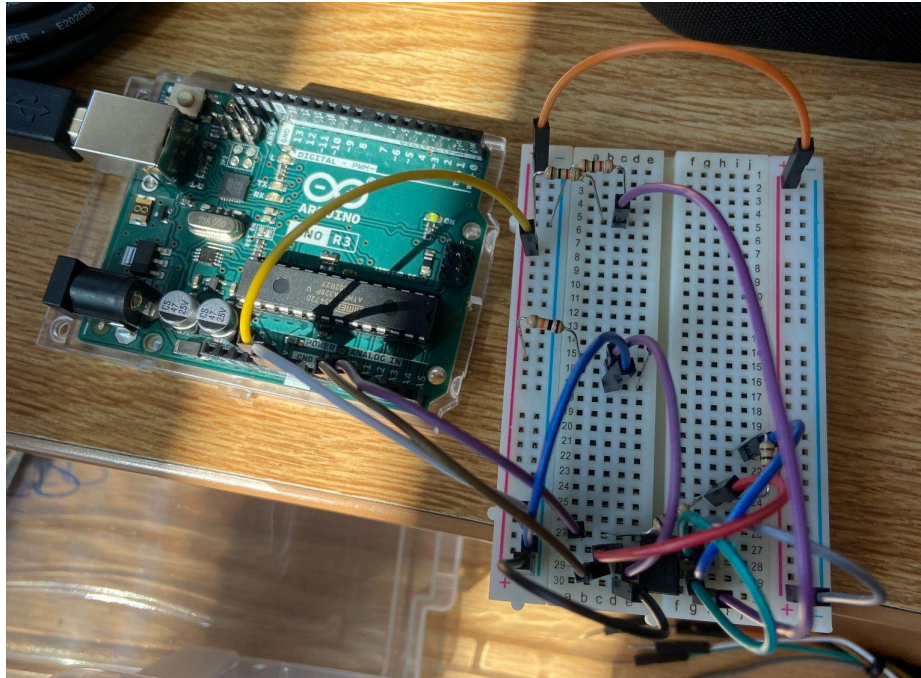
// Calculate temperature using Steinhart-Hart equation for NTC thermistors
float steinhart = log(resistance / R0);
steinhart /= B;
steinhart += 1.0 / (T0 + 273.15);
steinhart = 1.0 / steinhart;
steinhart -= 273.15;
return steinhart;
}

float calculateAmpTemperature(float amplifiedresistance) {
// Calculate temperature using Steinhart-Hart equation for NTC thermistors
float steinhart = log(amplifiedresistance / R0);
steinhart /= B;
steinhart += 1.0 / (T0 + 273.15);
steinhart = 1.0 / steinhart;
steinhart -= 273.15;
return steinhart;
}

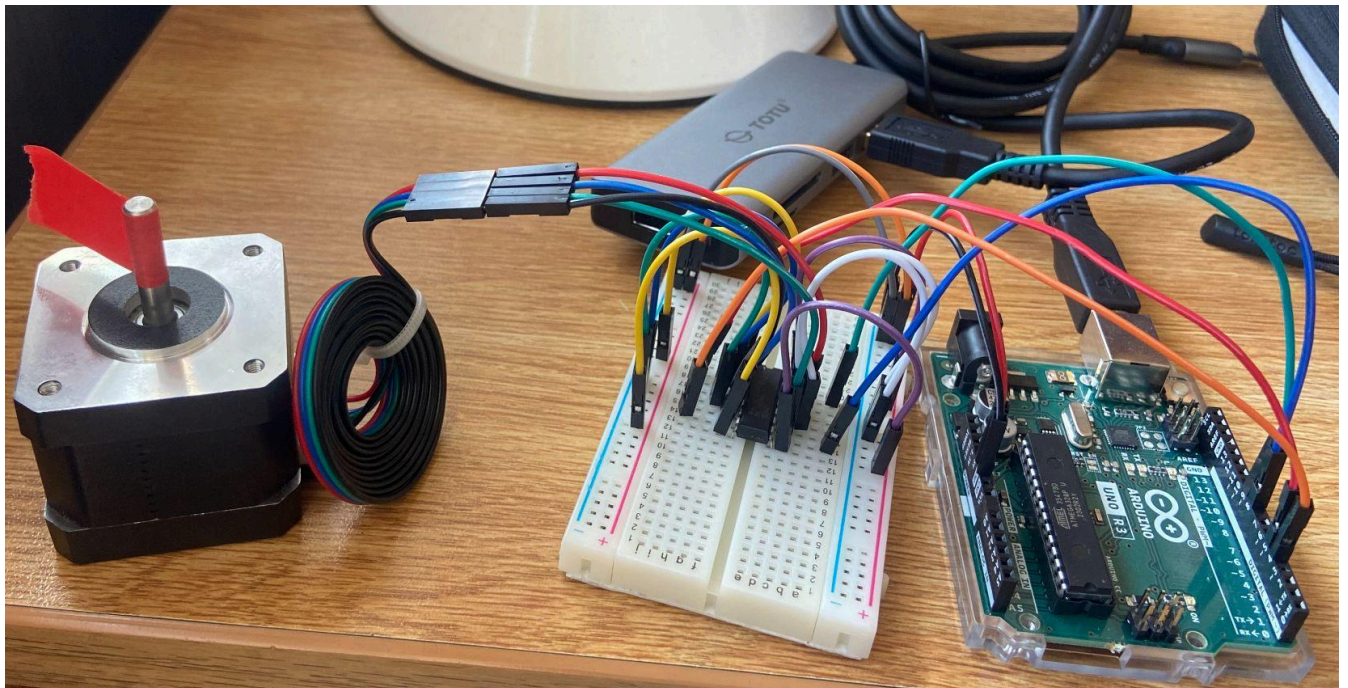
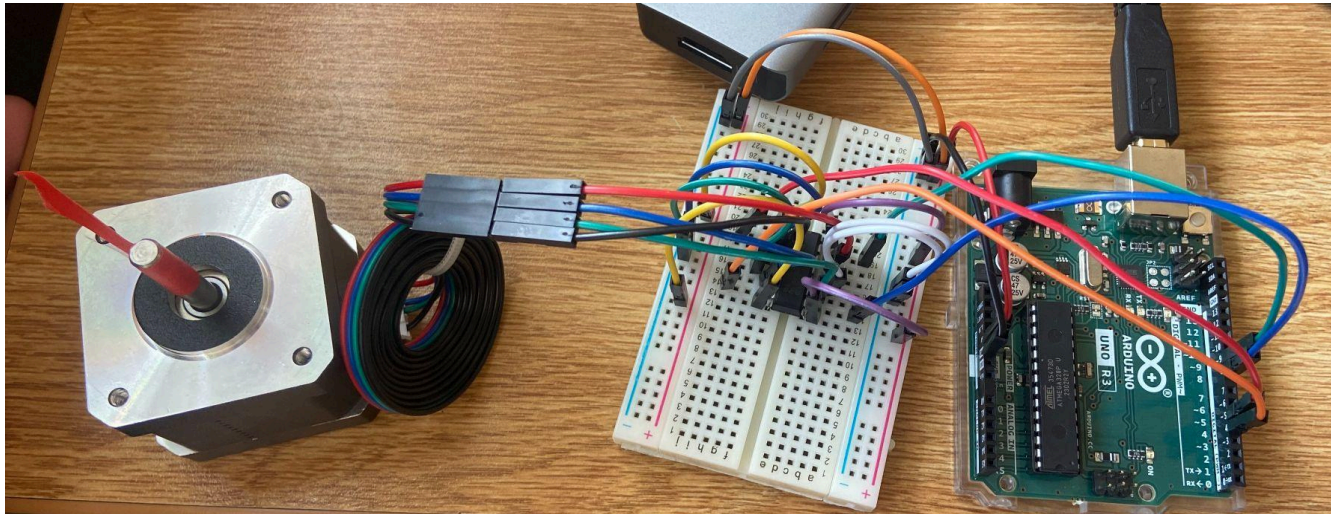
```

Write Up:

I did a similar approach in calculating the temperature of the unamplified voltage as before. I created another function that would calculate my new variables because of this amplified voltage. There is a difference because when we amplify the voltage we get a more zoomed in range of values, this allows us to be more precise as the range of values decreases but in return we cannot measure as high or low of voltages as before (since we zoomed in by amplification).



HW 3: Circuit Design/Setup:



a)

```
int counter = 0;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(8,OUTPUT);
  pinMode(9,OUTPUT);
  pinMode(4,OUTPUT);
  pinMode(5,OUTPUT);
}
```

```

digitalWrite(4,HIGH);
digitalWrite(5,LOW);
digitalWrite(8,HIGH);
digitalWrite(9,LOW);
delay(50);
}

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(8,LOW);
  digitalWrite(9,HIGH);
  counter++;
  delay(50);
  digitalWrite(4,LOW);
  digitalWrite(5,HIGH);
  counter++;
  delay(50);
  digitalWrite(8,HIGH);
  digitalWrite(9,LOW);
  counter++;
  delay(50);
  digitalWrite(4,HIGH);
  digitalWrite(5,LOW);
  counter++;
  delay(50);

  Serial.println(counter);
}

```

Write Up:

I used this code to count each step taken in the rotation of the stepper motor, which when a pin pair (4 and 5 or 8 and 9) switch from high to low and low to high respectively, would turn the gear one notch which is one step. If we counted these steps with the counter code and by taping a flag onto the top of the motor, we would observe the flag make one full loop and then unplug the Arduino and see what the counter number was at. In this case, I found this stepper motor has 200 steps in one full rotation.

Serial Monitor:

```
36
40
44
48
52
56
60
64
68
72
76
80
84
88
92
96
100
104
108
112
116
120
124
128
132
136
140
144
148
152
156
160
164
168
172
176
180
184
188
192
196
200
```

b)

```
int counter = 0;
```

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(8,OUTPUT);
  pinMode(9,OUTPUT);
  pinMode(4,OUTPUT);
  pinMode(5,OUTPUT);

  digitalWrite(4,LOW);
  digitalWrite(5,HIGH);
  digitalWrite(8,LOW);
  digitalWrite(9,HIGH);
  delay(50);
}

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(8,LOW);
  digitalWrite(9,HIGH);
```

```

    counter++;
    delay(50);
    digitalWrite(4,HIGH);
    digitalWrite(5,LOW);
    counter++;
    delay(50);
    digitalWrite(8,HIGH);
    digitalWrite(9,LOW);
    counter++;
    delay(50);
    digitalWrite(4,LOW);
    digitalWrite(5,HIGH);
    counter++;
    delay(50);

    Serial.println(counter);
}

```

i)

```

const int stepsperrevolution = 200;
int counter = 0;
bool rotating = false;

void setup() {
    Serial.begin(9600);
    pinMode(8, OUTPUT);
    pinMode(9, OUTPUT);
    pinMode(4, OUTPUT);
    pinMode(5, OUTPUT);

    digitalWrite(4, LOW);
    digitalWrite(5, HIGH);
    digitalWrite(8, LOW);
    digitalWrite(9, HIGH);
    delay(50);
}

void loop() {
    if (Serial.available() > 0) {
        //input from the serial monitor
    }
}

```

```

    int degrees = Serial.parseInt();

    // Rotate the stepper motor by the inputted number of degrees
    rotateDegrees(degrees);
}
}

// Function to rotate the stepper motor by number of degrees
void rotateDegrees(int degrees) {
    int stepstomove = stepsperrevolution * degrees / 360;
    int direction = (degrees > 0) ? 1 : -1;

    // Start the rotation
    rotating = true;

    // Rotate the motor until the specified number of steps is reached
    for (int i = 0; i < abs(stepstomove); i++) {
        // Check if the desired number of steps has been reached
        if (i >= abs(stepstomove)) {
            break; // Stop the rotation
        }

        step(direction);
        delay(50);
        counter++;
    }

    // Rotation completed, set rotating to false
    rotating = false;
}

// Function to take one step in the specified direction
void step(int direction) {
    // Clockwise rotation
    digitalWrite(8, LOW);
    digitalWrite(9, HIGH);
    counter++;
    delay(50);
    digitalWrite(4, LOW);
    digitalWrite(5, HIGH);
    counter++;
    delay(50);
}

```

```

digitalWrite(8,HIGH);
digitalWrite(9,LOW);
counter++;
delay(50);
digitalWrite(4,HIGH);
digitalWrite(5,LOW);
counter++;
delay(50);
} else {
    // Counterclockwise rotation
    digitalWrite(8,LOW);
    digitalWrite(9,HIGH);
    counter++;
    delay(50);
    digitalWrite(4,LOW);
    digitalWrite(5,HIGH);
    counter++;
    delay(50);
    digitalWrite(8,HIGH);
    digitalWrite(9,LOW);
    counter++;
    delay(50);
    digitalWrite(4,HIGH);
    digitalWrite(5,LOW);
    counter++;
    delay(50);
}

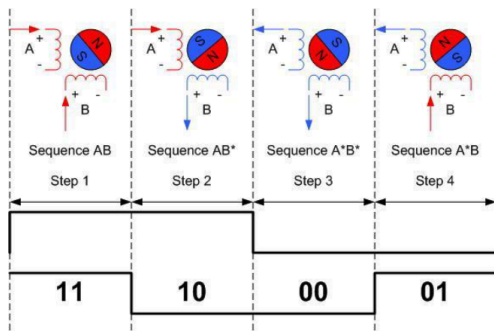
Serial.println(counter);
}

```

Write Up:

It was possible to make the motor turn backward we just needed to manipulate the order in which pins were turning on and off (chronological order matters here). We do this by inverting the pins' charges initially (making them the opposite of what they were in part a), and then inverting only one of the pins in the loop will reverse the spin of the motor (there are many methods of order

manipulation we can use to accomplish this). Doing this makes the magnetic field inside, produced by the two coils inside, activate in the reverse order.



So in part a it was moving like this because of its particular activation sequence, reversing this order sequence, as shown in the code reverses this turning making it spin counterclockwise (was spinning clockwise in part a).

i. I made a code to read an inputted degree (into the serial monitor) and turn it as much as it is put either CW for positive degrees or CCW for negative degrees. Coming up with a function and figuring out how to break the loop with it was pretty challenging. Limitations are that the minimum I could turn it is 4 steps minimum. I couldn't figure out how I could loop it and allow it to do only one step if I said 1.8 degrees, probably would need another if and break function.

2)

```
const int stepsperrevolution = 200;
int counter = 0;
bool rotating = false;

void setup() {
  Serial.begin(9600);
  pinMode(8, OUTPUT);
  pinMode(9, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);

  digitalWrite(4, LOW);
  digitalWrite(5, HIGH);
  digitalWrite(8, LOW);
  digitalWrite(9, HIGH);
  delay(50);
}
```

```

void loop() {
  if (Serial.available() > 0) {
    int degrees = Serial.parseInt();
    rotateDegrees(degrees);
  }
}

void rotateDegrees(int degrees) {
  int stepstomove = stepsperrevolution * degrees / 360;
  int direction = (degrees > 0) ? 1 : -1;
  rotating = true;

  for (int i = 0; i < abs(stepstomove); i++) {
    if (i >= abs(stepstomove)) {
      break;
    }

    step(direction);
    delay(50);
    counter++;
  }

  rotating = false;
}

void step(int direction) {
  if (direction == 1) {
    // Clockwise rotation
    digitalWrite(8, LOW);
    analogWrite(9, 127); // PWM at 50% duty cycle
    delay(50);
    digitalWrite(4, LOW);
    digitalWrite(5, HIGH);
    delay(50);
    digitalWrite(8, LOW);
    digitalWrite(9, HIGH);
    delay(50);
    digitalWrite(4, HIGH);
    digitalWrite(5, HIGH);
    delay(50);
  } else {
    // Counterclockwise rotation

```

```

    digitalWrite(8, LOW);
    digitalWrite(9, HIGH);
    delay(50);
    digitalWrite(4, LOW);
    digitalWrite(5, HIGH);
    delay(50);
    digitalWrite(8, LOW);
    analogWrite(9, 127); // PWM at 50% duty cycle
    delay(50);
    digitalWrite(4, HIGH);
    digitalWrite(5, HIGH);
    delay(50);
}
}

```

Write Up:

I activated PWM at 50% voltage for half steps at pin 9

3)

```

const int stepsPerRevolution = 200;
int counter = 0;
bool rotating = false;

void setup() {
    Serial.begin(9600);
    pinMode(8, OUTPUT);
    pinMode(9, OUTPUT);
    pinMode(10, OUTPUT);
    pinMode(11, OUTPUT);
    pinMode(4, OUTPUT);
    pinMode(5, OUTPUT);

    digitalWrite(4, LOW);
    digitalWrite(5, HIGH);
    digitalWrite(8, LOW);
    digitalWrite(9, HIGH);
    digitalWrite(10, LOW);
    digitalWrite(11, HIGH);
    delay(50);
}

void loop() {

```

```

    if (Serial.available() > 0) {
        int degrees = Serial.parseInt();
        rotateDegrees(degrees);
    }
}

void rotateDegrees(int degrees) {
    int stepsToMove = stepsPerRevolution * degrees / 360;
    int direction = (degrees > 0) ? 1 : -1;
    rotating = true;

    for (int i = 0; i < abs(stepsToMove); i++) {
        if (i >= abs(stepsToMove)) {
            break;
        }

        step(direction);
        delay(50);
        counter++;
    }

    rotating = false;
}

void step(int direction) {
    if (direction == 1) {
        // Clockwise rotation
        digitalWrite(8, LOW);
        analogWrite(9, 63); // PWM at 25% duty cycle
        digitalWrite(10, LOW);
        digitalWrite(11, HIGH);
        delay(50);
        digitalWrite(4, LOW);
        digitalWrite(5, HIGH);
        delay(50);
        digitalWrite(8, LOW);
        digitalWrite(9, HIGH);
        delay(50);
        digitalWrite(10, HIGH);
        digitalWrite(11, LOW);
        delay(50);
    } else {

```

```
// Counterclockwise rotation
digitalWrite(8, LOW);
digitalWrite(9, HIGH);
delay(50);
digitalWrite(4, LOW);
digitalWrite(5, HIGH);
delay(50);
digitalWrite(8, LOW);
analogWrite(9, 63); // PWM at 25% duty cycle
digitalWrite(10, HIGH);
digitalWrite(11, LOW);
delay(50);
digitalWrite(4, HIGH);
digitalWrite(5, HIGH);
delay(50);
}
}
```

Write Up:

I activated PWM at 25% voltage for half steps at pin 9

Processsing Code:

```
import processing.serial.*;
import controlP5.*;
```

```
Serial myPort; // Create object from Serial class
ControlP5 cp5;
```

```
float player1;
float player2;
```

```
float size1 = 50; // Set initial value for size1
float size2 = 50; // Set initial value for size2
```

```
float xpos;
float ypos;
```

```
float xspeed = 2;
float yspeed = 2;
```

```
int score1 = 0;
int score2 = 0;
```

```
boolean serving = false;
```

```
Slider p1slider;
Slider p2slider;
Button servebutton;
```

```
void setup()
{
  size(800,800);
  String portName = Serial.list()[0];
  myPort = new Serial(this, "/dev/cu.usbmodem1301", 115200); //connects to RP2040
  myPort.bufferUntil('\n');

  player1 = height / 2;
  player2 = height / 2;

  cp5 = new ControlP5(this);
```

```
p1slider = cp5.addSlider("Paddle 1 Size")
  .setPosition(10, 10)
  .setRange(10, 100)
  .setValue(size1)
  .setSize(100, 20);
```

```
p2slider = cp5.addSlider("Paddle 2 Size")
  .setPosition(400, 10)
  .setRange(10, 100)
  .setValue(size2)
  .setSize(100, 20);
```

```
servebutton = cp5.addButton("serveBall")
  .setPosition(350, 60)
  .setSize(100, 50)
  .setLabel("Serve");
resetBall();
}
```

```
void draw()
{
  background(255);
  fill(0);
  rect(0, player1-37.5, 10, size1);
  rect(790, player2-37.5, 10, size2);
```

```
ellipse(xpos, ypos, 20, 20);
```

```
if (serving) {
  // Update ball position
  xpos += xspeed;
  ypos += yspeed;

  // Check for collision with top/bottom walls
  if (ypos <= 0 || ypos >= height) {
    yspeed *= -1;
  }
}
```

```

// Check for collision with paddles
if (xpos <= 10 && abs(ypos - player1) <= 37.5) {
  xspeed *= -1;
}
if (xpos >= width - 10 && abs(ypos - player2) <= 37.5) {
  xspeed *= -1;
}

// Check if player 1 scored
if (xpos < 0 ) {
  // Reset ball position
  score1++;
  serving = false;
  resetBall();
}
// Check if player 2 scored
if (xpos > width) {
  score2++;
  // Reset ball position
  serving = false;
  resetBall();
}
}

textAlign(CENTER);
textSize(32);
fill(0);
text(score1, width / 4, 50);
text(score2, width * 3 / 4, 50);
}

void serialEvent(Serial port){
  String inputString = port.readStringUntil('\n');
  if (inputString != null) {
    String[] data = inputString.trim().split(" ");
    if (data.length >= 4) {
      float temp = float(data[1]);
      float temp2 = float(data[3]);
      player1 = map(temp, 0, 4095, 0, height);
      player2 = map(temp2, 0, 4095, 0, height);
    }
  }
}

```

```

        player2 = height - player2;
        println("Height", player1);
        println("Height", player2);
    }
}
}

```

```

void resetBall(){
    xpos = random(20, width - 20);
    ypos = random(20, height - 20);
}

```

```

void size1(float val) {
    size1 = val;
}

```

```

void size2(float val) {
    size2 = val;
}

```

```

void serveBall() {
    serving = true;
}

```

Arduino Code:

```

void setup() {
    // put your setup code here, to run once:
    Serial.begin(9600);
    pinMode(A0, INPUT);
    pinMode(A1, INPUT);
}

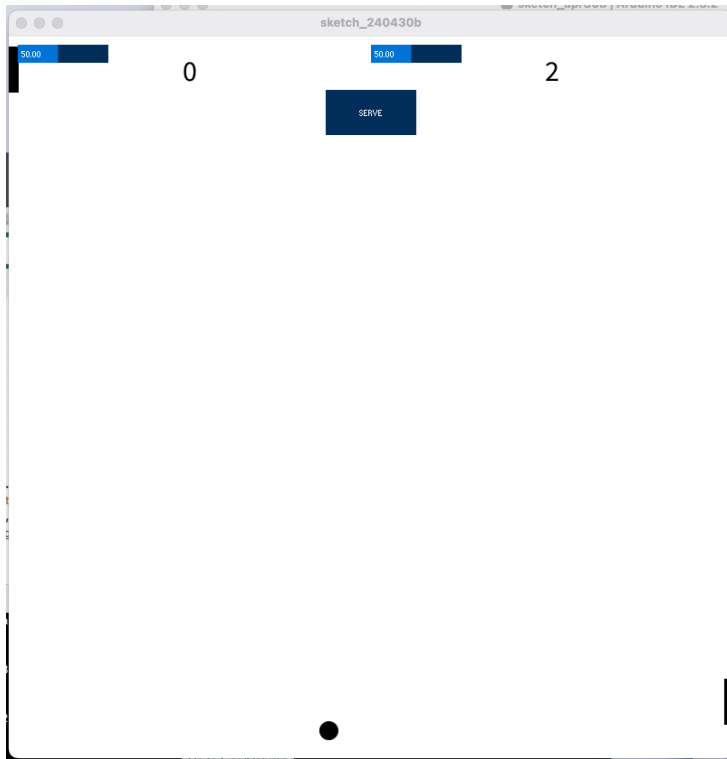
void loop() {
    // put your main code here, to run repeatedly:
    int player1 = analogRead(A0);
    int player2 = analogRead(A1);

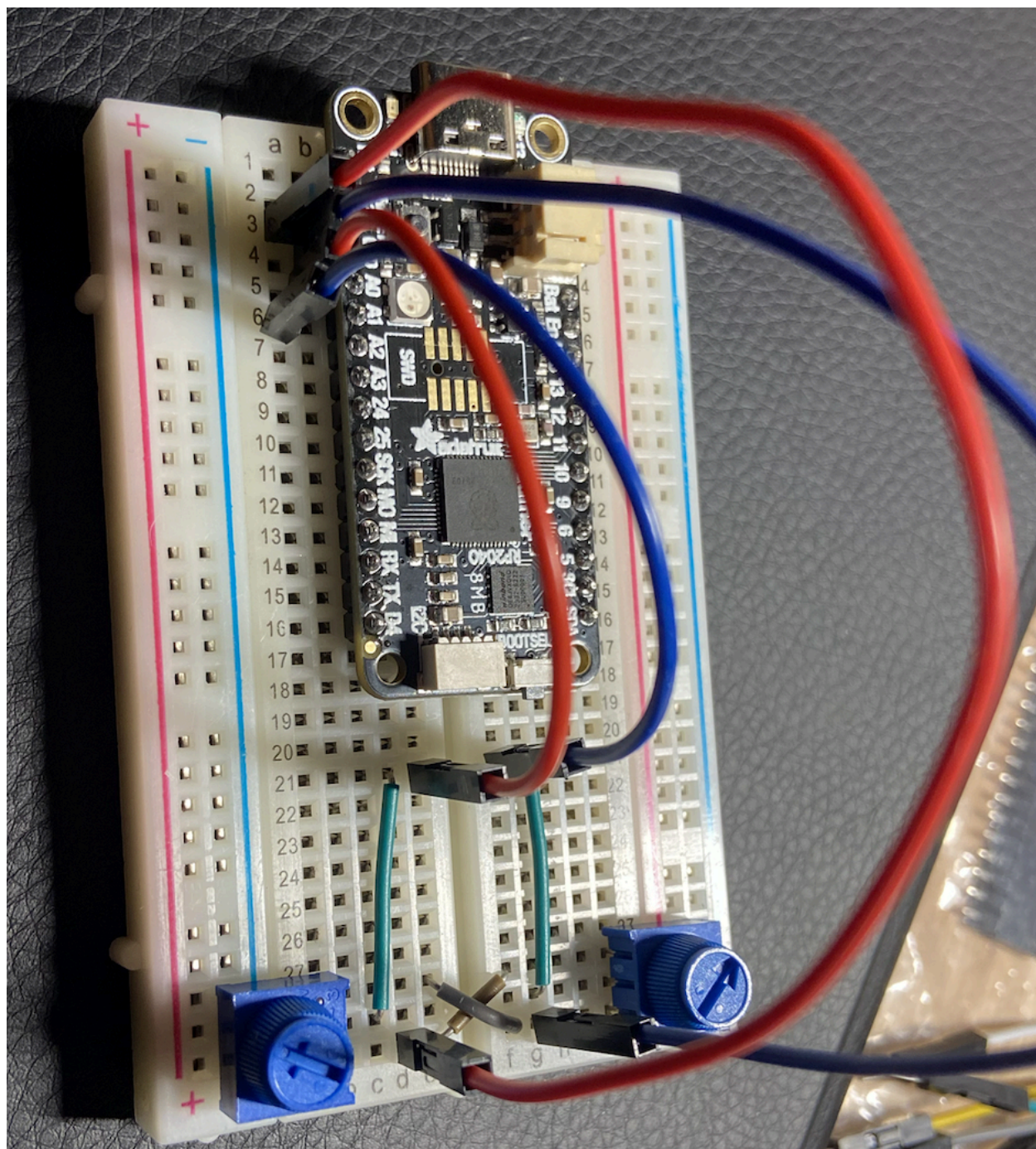
    Serial.print(player1);
    Serial.print(" 255");
    Serial.print(" ");
    Serial.print(player2);
}

```

```
Serial.println(" 255");  
  
delay(100);  
  
}
```

Screenshot of Pong:

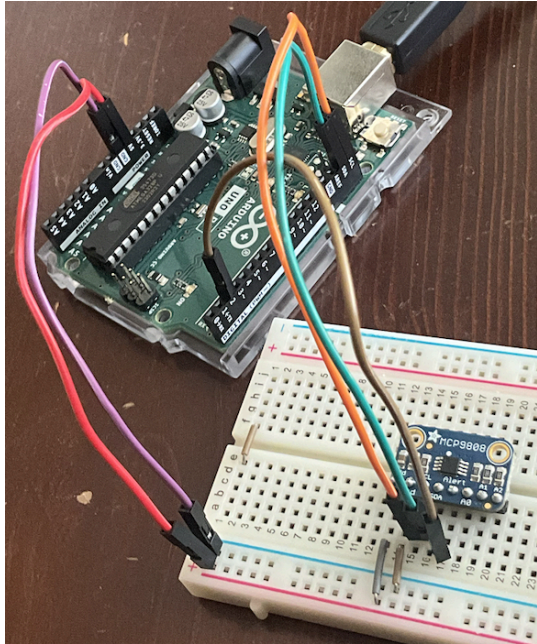




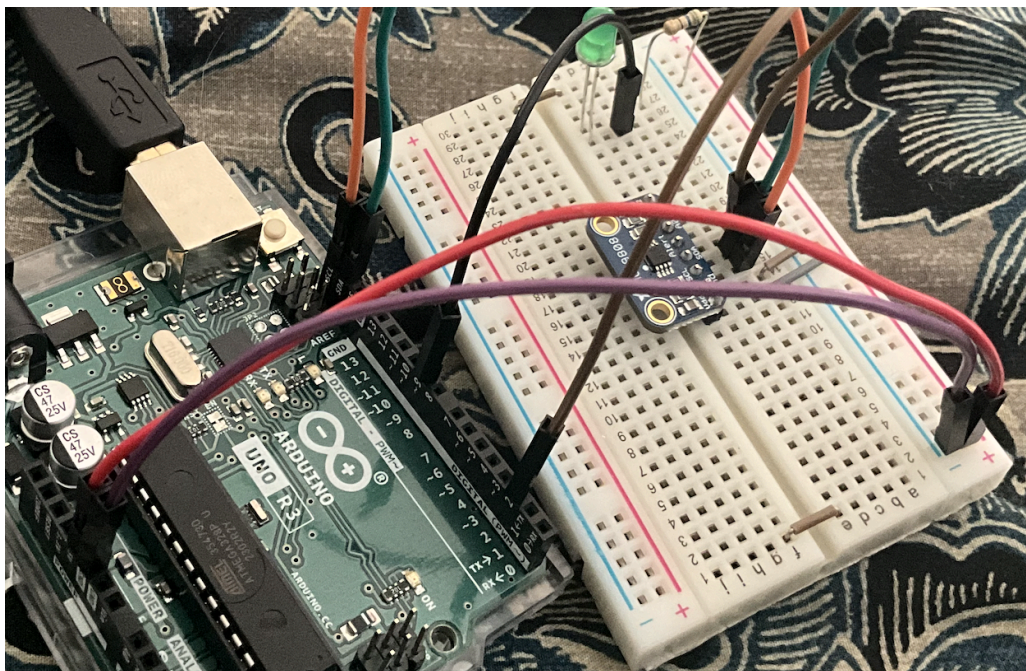
HW 5:

Board Setup:

Part 1) Note: Didn't use the alert pin for part 1 only part 2.



Part 2)



Part 1 - Reading temperature over I2C using the MCP 9808:

a)

```
//1a
```

```
#include <Wire.h>
```

```

#define MCP9808_I2CADDR_DEFAULT 0x18 // The MCP9808 default I2C address

void setup() {
  Serial.begin(9600);
  Wire.begin();
  Wire.beginTransmission(MCP9808_I2CADDR_DEFAULT);
  Wire.write(0x01); // Pointer register to Config register
  Wire.write(0x00); // Set Config register to continuous conversion mode
  Wire.write(0x00);
  Wire.endTransmission();
}

void loop() {

  float temperature = readTemperature();
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  // Delay for 10 ms, 100 ms, or 1000 ms
  delay(1000); // 10 ms delay
  // delay(100); // 100 ms delay
  // delay(1000); // 1000 ms delay
}

float readTemperature() {
  Wire.beginTransmission(MCP9808_I2CADDR_DEFAULT);
  Wire.write(0x05); // Pointer register to Temp register
  Wire.endTransmission();
  Wire.requestFrom(MCP9808_I2CADDR_DEFAULT, 2); // Request 2 bytes of data
  uint16_t data = 0;
  if (Wire.available() >= 2) {
    data = Wire.read() << 8 | Wire.read();
  }

  // Temp Data in Celsius
  float temp = (data & 0x0FFF) / 16.0;
  if (data & 0x1000) {
    temp -= 256;
  }
  return temp;
}

```

b)

```
//1b
#include <Wire.h>

#define MCP9808_I2CADDR_DEFAULT 0x18 // MCP9808 default I2C address

void setup() {
  Serial.begin(9600);
  Wire.begin();
  // Initialize MCP9808
  Wire.beginTransmission(MCP9808_I2CADDR_DEFAULT);
  Wire.write(0x01); // Pointer register to Config register
  Wire.write(0x60); // Set Config register to continuous conversion mode and 0.5°C
  resolution
  Wire.endTransmission();
}

void loop() {
  // Read temp from sensor
  float temperature = readTemperature();
  // Print temp
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  // Delay for 10 ms, 100 ms, or 1000 ms
  delay(10); // 10 ms delay
  // delay(100); // 100 ms delay
  // delay(1000); // 1000 ms delay
}

float readTemperature() {
  Wire.beginTransmission(MCP9808_I2CADDR_DEFAULT);
  Wire.write(0x05); // Pointer register to Temp register
  Wire.endTransmission();
  Wire.requestFrom(MCP9808_I2CADDR_DEFAULT, 2); // Request 2 bytes of data
  uint16_t data = 0;
  if (Wire.available() >= 2) {
    data = Wire.read() << 8 | Wire.read();
  }
}
```

```
// Temp Data in Celsius with 0.5°C resolution
float temp = (data & 0x0FFF) / 2.0;
if (data & 0x1000) {
temp -= 256;
}
return temp;
}
```

Part 2 - Setting an Interrupt:

```
#include <Wire.h>

#define MCP9808_I2CADDR_DEFAULT 0x18 // MCP9808 default I2C address
#define MCP9808_REG_CONFIG 0x01
#define MCP9808_REG_TUPPER 0x02
#define MCP9808_REG_TLOWER 0x03
#define MCP9808_REG_TCRIT 0x04
#define MCP9808_REG_ALERT_CONFIG 0x08

#define LED_PIN 9 // LED Pin
#define ALERT_PIN 2 // interrupt pin

volatile bool alertTriggered = false;

void setup() {
  Serial.begin(9600);
  Wire.begin();
  // Configure MCP9808
  Wire.beginTransmission(MCP9808_I2CADDR_DEFAULT);
  Wire.write(MCP9808_REG_TUPPER);
  Wire.write(3); // Tupper to 3°C above room temp
  Wire.write(0);
  Wire.endTransmission();
  Wire.beginTransmission(MCP9808_I2CADDR_DEFAULT);
  Wire.write(MCP9808_REG_TLOWER);
  Wire.write(-2); // Tlower to 2°C below room temp
  Wire.write(0);
  Wire.endTransmission();
  Wire.beginTransmission(MCP9808_I2CADDR_DEFAULT);
  Wire.write(MCP9808_REG_TCRIT);
  Wire.write(128); // Set Tcrit to 128°C
  Wire.write(0);
}
```

```

Wire.endTransmission();

// Configure alert pin as interrupt
Wire.beginTransmission(MCP9808_I2CADDR_DEFAULT);
Wire.write(MCP9808_REG_ALERT_CONFIG);
Wire.write(0x10); // Active low, interrupt mode
Wire.endTransmission();

// Attaching interrupt to alert pin
attachInterrupt(digitalPinToInterrupt(ALERT_PIN), handleInterrupt, FALLING);

// Setup LED pin
pinMode(LED_PIN, OUTPUT);
}

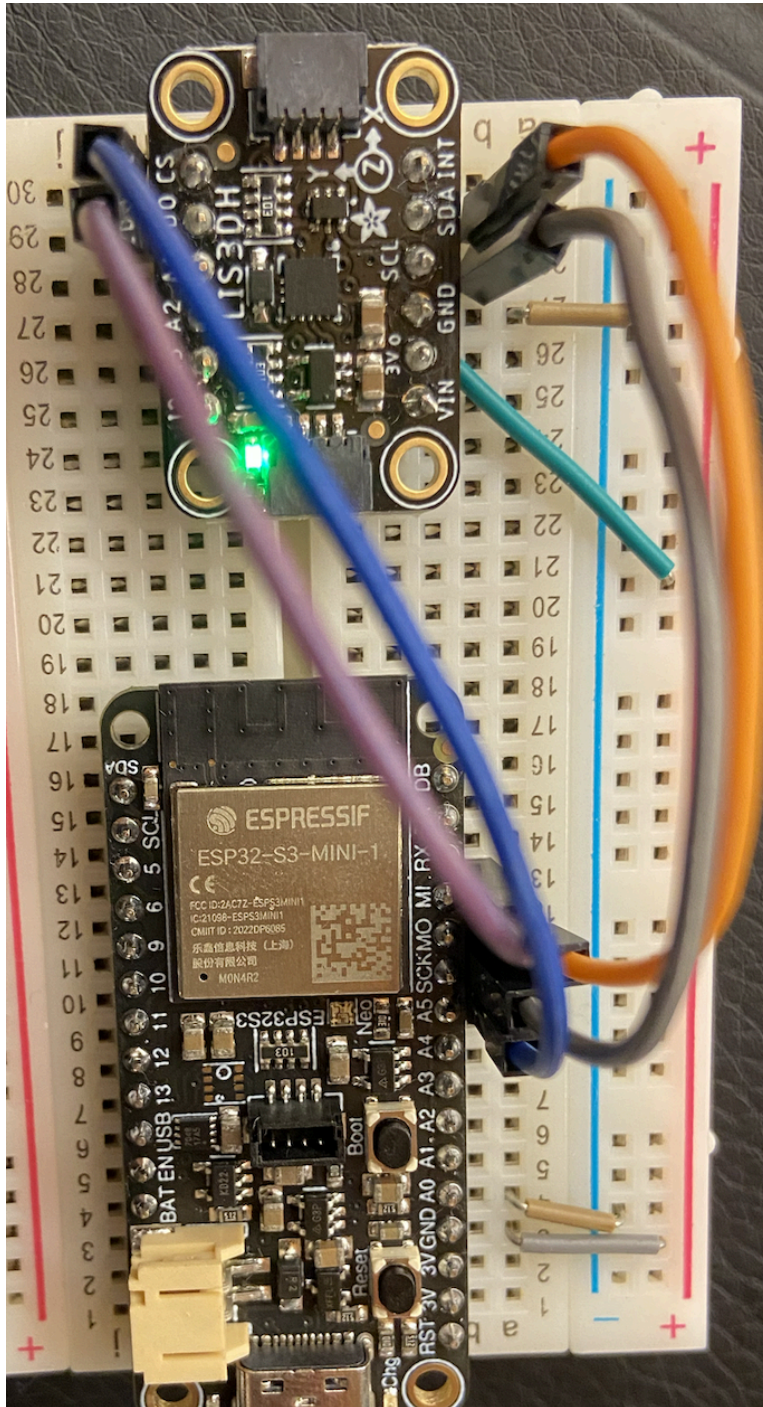
void loop() {
  if (alertTriggered) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("Temperature threshold crossed!");
    alertTriggered = false;
  } else {
    digitalWrite(LED_PIN, LOW);
  }
  delay(1000);
}

void handleInterrupt() {
  alertTriggered = true;
}

```

Write Ups In the Comment Section of Assignment HW5!

HW 6:



Part 1:

ino tab:

```
#include <SPI.h>
```

```
#include "myAccelLib.h"
```

```
int CSpin =A5;
```

```

void setup() {
  Serial.begin(9600);
  while(!Serial);
  pinMode(CSpin, OUTPUT);
  SPI.begin(); // initialize SPI:
  configChip(CSpin);

  /*digitalWrite(CSpin, LOW);
  SPI.transfer(0b10001111); //Read from WHOAMI
  byte ourData = SPI.transfer(0b11111111);
  digitalWrite(CSpin, HIGH);
  Serial.print("WHOAMI? ");
  Serial.println(ourData, BIN);
  */

}

void loop() {
  xAccel(CSpin);
  yAccel(CSpin);
  zAccel(CSpin);
  /*
  digitalWrite(CSpin, LOW);
  SPI.transfer(0b11101000); //Read multiple bytes starting with OUT_X_L
  byte XLB = SPI.transfer(0b00000000);
  byte XHB = SPI.transfer(0b00000000);
  byte YLB = SPI.transfer(0b00000000);
  byte YHB = SPI.transfer(0b00000000);
  byte ZLB = SPI.transfer(0b00000000);
  byte ZHB= SPI.transfer(0b00000000);
  digitalWrite(CSpin, HIGH);

  int16_t xAccel = (XHB << 8) + XLB;
  int16_t yAccel = (YHB << 8) + YLB;
  int16_t zAccel = (ZHB << 8) + ZLB;

  xAccel /= 16;
  yAccel /= 16;
  zAccel /= 16;
  */
}

```

```

Serial.print(xAccel/2048.0*2.0);
Serial.print(" ");
Serial.print(yAccel/2048.0*2.0);
Serial.print(" ");
Serial.println(zAccel/2048.0*2.0);
*/
delay(100);
}

```

.cpp tab:

```

#include <Arduino.h>
#include <SPI.h>

void configChip(int CSpin) {
  SPI.begin(); // initialize SPI:
  digitalWrite(CSpin,LOW);
  SPI.transfer(0b00100000); //CTRLREG1
  SPI.transfer(0b01110111); // 400 Hz XYZ axes enabled
  digitalWrite(CSpin,HIGH);
  digitalWrite(CSpin,LOW);
  SPI.transfer(0b00100011); //CTRLREG4
  SPI.transfer(0b10001000); // Block Data update, 2G scale, High resolution
  digitalWrite(CSpin,HIGH);
}

void xAccel (int CSpin) {
  digitalWrite(CSpin,LOW);
  SPI.transfer(0b11101000);
  byte XLB = SPI.transfer(0b00000000);
  byte XHB = SPI.transfer(0b00000000);
  digitalWrite(CSpin,HIGH);
  int16_t xAccel = (XHB << 8) + XLB;
  xAccel /= 16;
  Serial.print(xAccel/2048.0*2.0);
  Serial.print(" ");
}

void yAccel (int CSpin) {
  digitalWrite(CSpin,LOW);
  SPI.transfer(0b11101010);
  byte YLB = SPI.transfer(0b00000000);

```

```

byte YHB = SPI.transfer(0b00000000);
digitalWrite(CSpin,HIGH);
int16_t yAccel = (YHB << 8) + YLB;
yAccel /= 16;
Serial.print(yAccel/2048.0*2.0);
Serial.print(" ");
}

void zAccel (int CSpin) {
digitalWrite(CSpin,LOW);
SPI.transfer(0b11101100);
byte ZLB = SPI.transfer(0b00000000);
byte ZHB = SPI.transfer(0b00000000);
digitalWrite(CSpin,HIGH);
int16_t zAccel = (ZHB << 8) + ZLB; //how is order determined?
zAccel /= 16;
Serial.println(zAccel/2048.0*2.0);
}

```

.h tab:

```

void configChip(int CSpin);
void xAccel(int CSpin);
void yAccel(int CSpin);
void zAccel(int CSpin);

```

Part 2:

Processing code:

```
import processing.serial.*;
```

```

Serial sPort;
float aX, aY, aZ;
float rX, rY, rZ;
String[] sData = new String[3];
PImage[] imgs = new PImage[6];

```

```

void setup() {
  size(800, 800, P3D);
  sPort = new Serial(this, "/dev/cu.usbmodem21401", 9600);
  sPort.bufferUntil('\n');
  loadImgs();
}

```

```
}
```

```
void draw() {  
  background(255);  
  translate(width / 2, height / 2);  
  applyRotation(rX, rY, rZ);  
  drawCube();  
}
```

```
void serialEvent(Serial port) {  
  processSerialData(port.readStringUntil('\n'));  
}
```

```
void processSerialData(String input) {  
  input = trim(input);  
  String[] parsedData = split(input, ' ');  
  if (parsedData.length == 3) {  
    for (int i = 0; i < 3; i++) {  
      sData[i] = parsedData[i];  
    }  
    updateAccelValues();  
  }  
}
```

```
void updateAccelValues() {  
  aX = parseFloat(sData[0]);  
  aY = parseFloat(sData[1]);  
  aZ = parseFloat(sData[2]);  
  rX = map(aX, -2048, 2048, -PI, PI);  
  rY = map(aY, -2048, 2048, -PI, PI);  
  rZ = map(aZ, -2048, 2048, -PI, PI);  
  println(rX);  
}
```

```
void applyRotation(float x, float y, float z) {  
  rotateX(x);  
  rotateY(y);  
}
```

```
    rotateZ(z);  
}
```

```
void loadImgs() {  
    for (int i = 0; i < 6; i++) {  
        imgs[i] = loadImage("image" + (i + 1) + ".png");  
    }  
}
```

```
void drawCube() {  
    float sz = 200;  
    float hsz = sz / 2;
```

```
    beginShape();  
    texture(imgs[0]);  
    vertex(-hsz, -hsz, hsz, 0, 0);  
    vertex(hsz, -hsz, hsz, 1, 0);  
    vertex(hsz, hsz, hsz, 1, 1);  
    vertex(-hsz, hsz, hsz, 0, 1);  
    endShape(CLOSE);
```

```
    beginShape();  
    texture(imgs[1]);  
    vertex(hsz, -hsz, -hsz, 0, 0);  
    vertex(-hsz, -hsz, -hsz, 1, 0);  
    vertex(-hsz, hsz, -hsz, 1, 1);  
    vertex(hsz, hsz, -hsz, 0, 1);  
    endShape(CLOSE);
```

```
    beginShape();  
    texture(imgs[2]);  
    vertex(hsz, -hsz, hsz, 0, 0);  
    vertex(hsz, -hsz, -hsz, 1, 0);  
    vertex(hsz, hsz, -hsz, 1, 1);  
    vertex(hsz, hsz, hsz, 0, 1);  
    endShape(CLOSE);
```

```
beginShape();  
texture(imgs[3]);  
vertex(-hsz, -hsz, -hsz, 0, 0);  
vertex(-hsz, -hsz, hsz, 1, 0);  
vertex(-hsz, hsz, hsz, 1, 1);  
vertex(-hsz, hsz, -hsz, 0, 1);  
endShape(CLOSE);
```

```
beginShape();  
texture(imgs[4]);  
vertex(-hsz, -hsz, -hsz, 0, 0);  
vertex(hsz, -hsz, -hsz, 1, 0);  
vertex(hsz, -hsz, hsz, 1, 1);  
vertex(-hsz, -hsz, hsz, 0, 1);  
endShape(CLOSE);
```

```
beginShape();  
texture(imgs[5]);  
vertex(-hsz, hsz, hsz, 0, 0);  
vertex(hsz, hsz, hsz, 1, 0);  
vertex(hsz, hsz, -hsz, 1, 1);  
vertex(-hsz, hsz, -hsz, 0, 1);  
endShape(CLOSE);
```

```
}
```



AccelerometerCu
be3D.pde

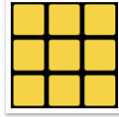


image5.jpg 
Error

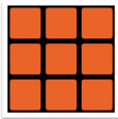


image1.jpg 
Error



image2.jpg 
Error



image3.jpg 
Error

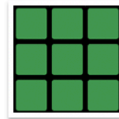


image4.jpg 
Error

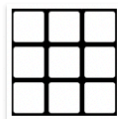
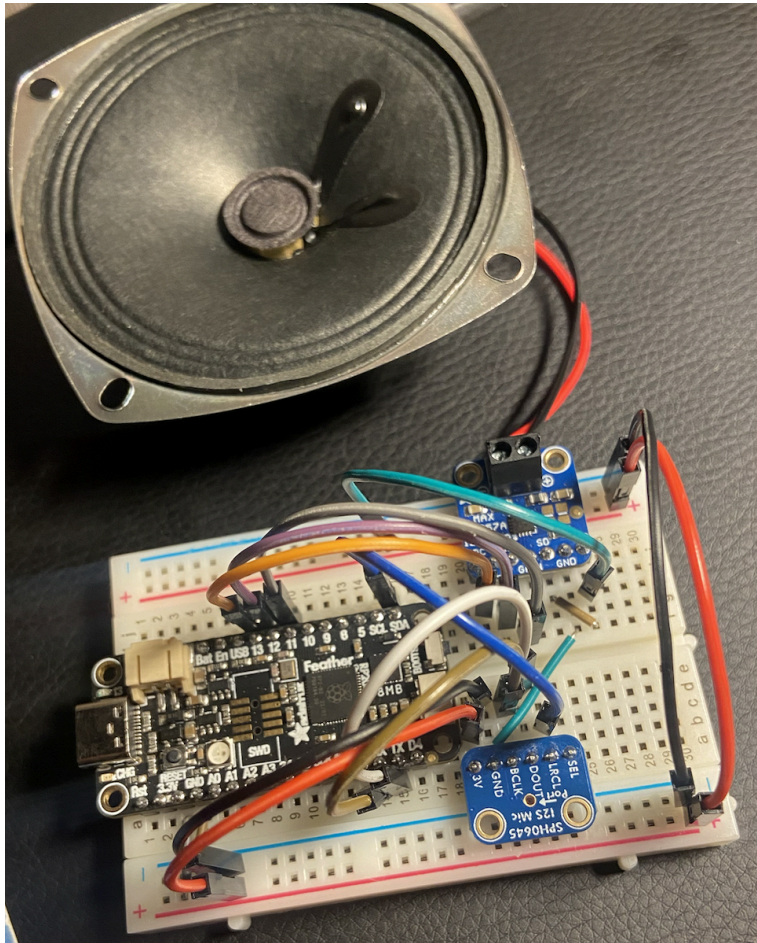


image6.jpg 
Error

Write Ups In the Comment Section of Assignment HW6!

HW 7:

Board Setup:



Final Code:

```
#include <I2S.h>

int volume = 7;
const int buttonPin = 8;

I2S input(INPUT);
I2S output(OUTPUT);

int32_t oldval = 0;
int32_t oldfiltval = 0;

const int maxsamps = 65536;
int16_t recorddata[maxsamps] = {0};
uint32_t curntidx = 0;
```

```

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT_PULLUP);
  input.setDATA(0);
  input.setBCLK(1);
  input.setBitsPerSample(32);
  input.setFrequency(16000);
  input.begin();

  output.setBCLK(11);
  output.setDATA(10);
  output.setBitsPerSample(32);

  int samprate = 16000;
  if (!output.begin(samprate)) {
    Serial.println("Failed to initialize output I2S!");
    while (1);
  }
}

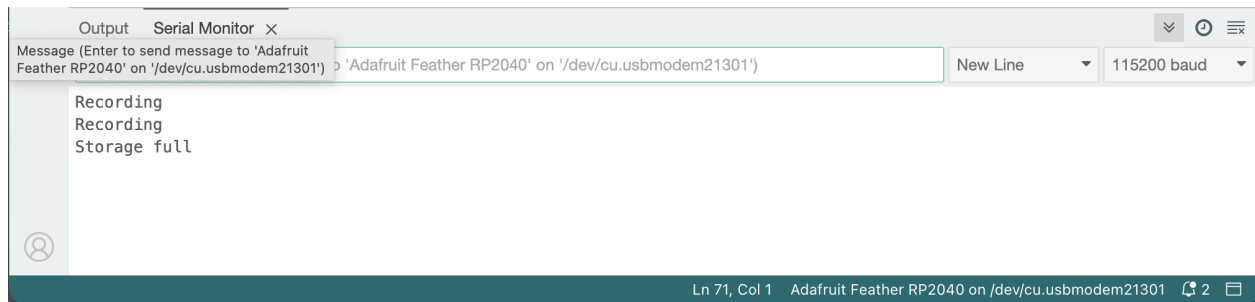
void loop() {
  int32_t oldval = 0;
  int32_t oldfiltval = 0;
  int16_t counter = 0;
  int16_t recorddata[65536];

  if (digitalRead(buttonPin) == LOW) {
    Serial.println("Recording");
    delay(500);
    for (uint32_t index = currtidx; index < maxsamps; index++) {
      int32_t left, right;
      input.read32(&left, &right);
      left >>= 14;
      int16_t shftlft = left >> 2;
      recorddata[index] = shftlft;
      currtidx = index + 1;
    }
    // Audio playback
    while (digitalRead(buttonPin) == LOW) {
      for (uint32_t a = 0; a < currtidx; a++) {
        output.write32(volume * recorddata[a] << 16, volume * recorddata[a] << 16);
      }
    }
  }
}

```

```
}  
}  
}
```

Serial Monitor:



Write Up In the Comment Section of Assignment HW7!